

CS3354 Software Engineering

Deliverable #2

1. Title of Project

The title of our project is **Study Buddy Finder**. Study Buddy Finder is a software application designed to help students find study partners, create study groups, and schedule study sessions based on shared courses and academic interests.

2. Delegation of Tasks

Diego Chapa – Responsible for creating the Software Test Plan, including defining test cases, testing strategies, expected results, and verifying that the system functions correctly.

Raisa Reza – Responsible for designing the User Experience (UX) Prototype, including the application's interface, screen layouts, navigation flow, and overall user interaction.

Rafia Rauf – Responsible for developing the Project Schedule, including the project timeline, milestones, task breakdown structure (WBS), effort estimation, cost estimation, and overall project planning documentation.

Shobhit Kapoor – Responsible for implementing the Backend of the Study Buddy Finder application, including database connectivity, server-side logic, APIs, and core application functionality.

Vladislav Kondratyev – Responsible for conducting a Comparative Analysis between the Study Buddy Finder application and an existing application in the same domain. This includes identifying similarities, differences, strengths, weaknesses, and features that can improve the proposed system.

3. Deliverable #1

1. Title of Project The title of our project is Study Buddy Finder. Study Buddy Finder is a software application designed to help students find study partners, create study groups, and schedule study sessions based on shared courses and academic interests.

2. Delegation of Tasks The following tasks have been assigned to team members:

- Diego Chapa – Create the first commit to the GitHub repository.
- Rafia Rauf – Create the Use Case Diagram and Sequence Diagrams.
- Raisa Reza – Create the Architectural Design Diagram and Class Diagram.
- Shobhit Kapoor – Create the second GitHub commit containing the Project Scope document.
- Vladislav Kondratyev – Create the GitHub repository and add all team members and the TA as collaborators.

Each team member is contributing to the completion of Deliverable #1 according to the responsibilities listed above.

3. Assumptions

The following assumptions are made for the Study Buddy Finder system:

- Users must register and create an account before accessing the system.
- Each student will provide accurate course information when creating their profile.
- The system will only be used by students enrolled at the university.
- Internet access is required to use the application.
- Students may participate in multiple study groups at the same time.
- Study sessions may be conducted either in person or online.
- The system will not verify course enrollment with the university database.

- Users are responsible for maintaining appropriate and respectful communication with other students.

4. The feedback received for our project proposal stated that "The project plan appears to be sound." This indicates that the proposed project was well planned and did not require significant revisions.

Based on this feedback, our team will continue with the proposed Study Buddy Finder project as originally planned. We will further develop the system by refining the software requirements, UML diagrams, architectural design, and implementation details throughout the remaining project deliverables. We will also continue reviewing our work to ensure the project remains organized, complete, and consistent with the project objectives.

5. Our team has chosen the Agile software process model for the development of the Study Buddy Finder application. The Agile model is well suited for this project because it enables our team to divide work into smaller tasks, collaborate efficiently, and make improvements as new ideas or instructor feedback become available. Further, the Study Buddy Finder application contains multiple features, such as user registration, study group creation, partner matching, and study session scheduling, developing these features incrementally will help ensure that each component is carefully designed, tested, and refined before moving on to the next.

In addition, Agile promotes regular communication among team members, making it easier to coordinate responsibilities and monitor project progress throughout the semester. This approach provides the flexibility needed to adapt to any changes while maintaining a well-organized development process.

6.

6.a Functional Requirements The Study Buddy Finder shall:

FR1. Allow students to register and create a personal account.

FR2. Allow registered users to securely log in and log out of the application.

FR3. Allow students to create and edit their profiles, including courses, major, and study preferences.

FR4. Allow students to search for study partners based on shared courses or academic interests.

FR5. Allow students to create, join, and leave study groups.

FR6. Allow students to schedule study sessions by selecting a date, time, and meeting location or virtual meeting option.

FR7. Allow administrators to manage user accounts and remove inappropriate content or inactive accounts.

6.b Non-Functional Requirements

-Product Requirements

Usability Requirement

- The system shall provide an intuitive and user-friendly interface so that new users can learn to navigate the application within 10 minutes.

Performance Requirement

- The system shall display search results and load pages within 3 seconds under normal operating conditions.

Space Requirement

- The system shall support storage of at least 10,000 user profiles and study groups without affecting system performance.

Dependability Requirement

- The system shall maintain at least 99% availability during the academic semester.

Security Requirement

- User passwords shall be encrypted, and only authenticated users shall have access to their accounts.

-Organizational Requirements

Environmental Requirement

- The application shall operate on modern web browsers and mobile devices with an internet connection.

Operational Requirement

- The system shall be available 24 hours a day to allow students to access study resources at any time.

Development Requirement

- The application will be developed using Java, GitHub for version control, and MySQL for database management.

-External Requirements

Regulatory Requirement

- The system shall follow university policies regarding appropriate student conduct and responsible use of online services.

Ethical Requirement

- The application shall protect user privacy and shall not disclose personal information without the user's consent.

Legislative Requirement

- The system shall comply with applicable data privacy laws and regulations regarding the collection and storage of user information.

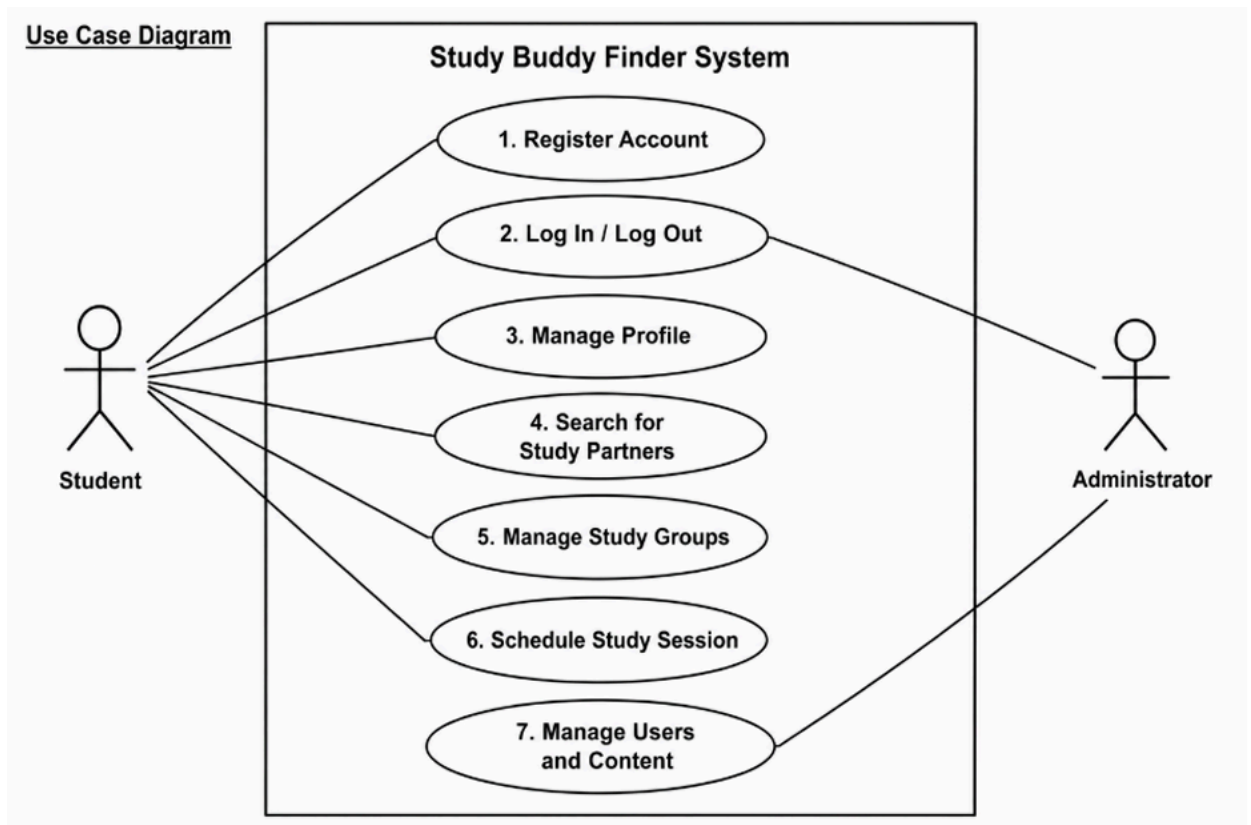
Accounting Requirement

- The system shall maintain logs of user account activities for administrative review when necessary.

Safety/Security Requirement

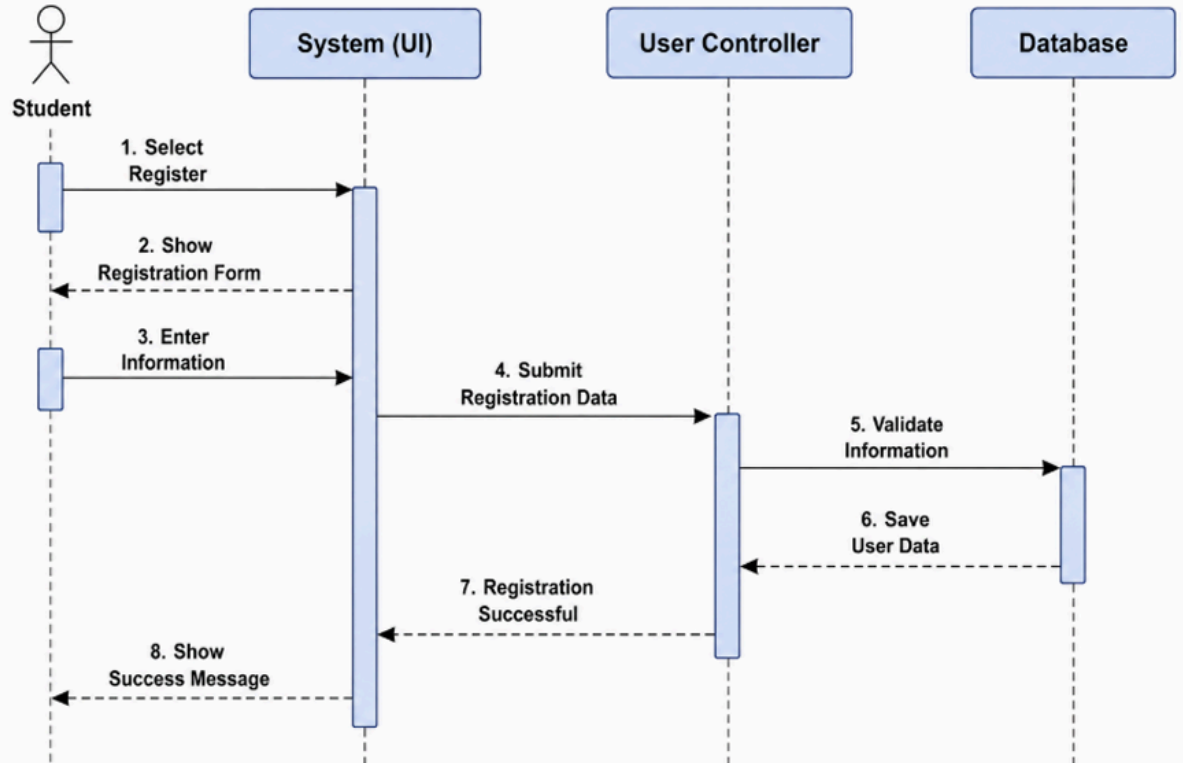
- The system shall automatically log users out after 30 minutes of inactivity.

7. Use Case Diagram

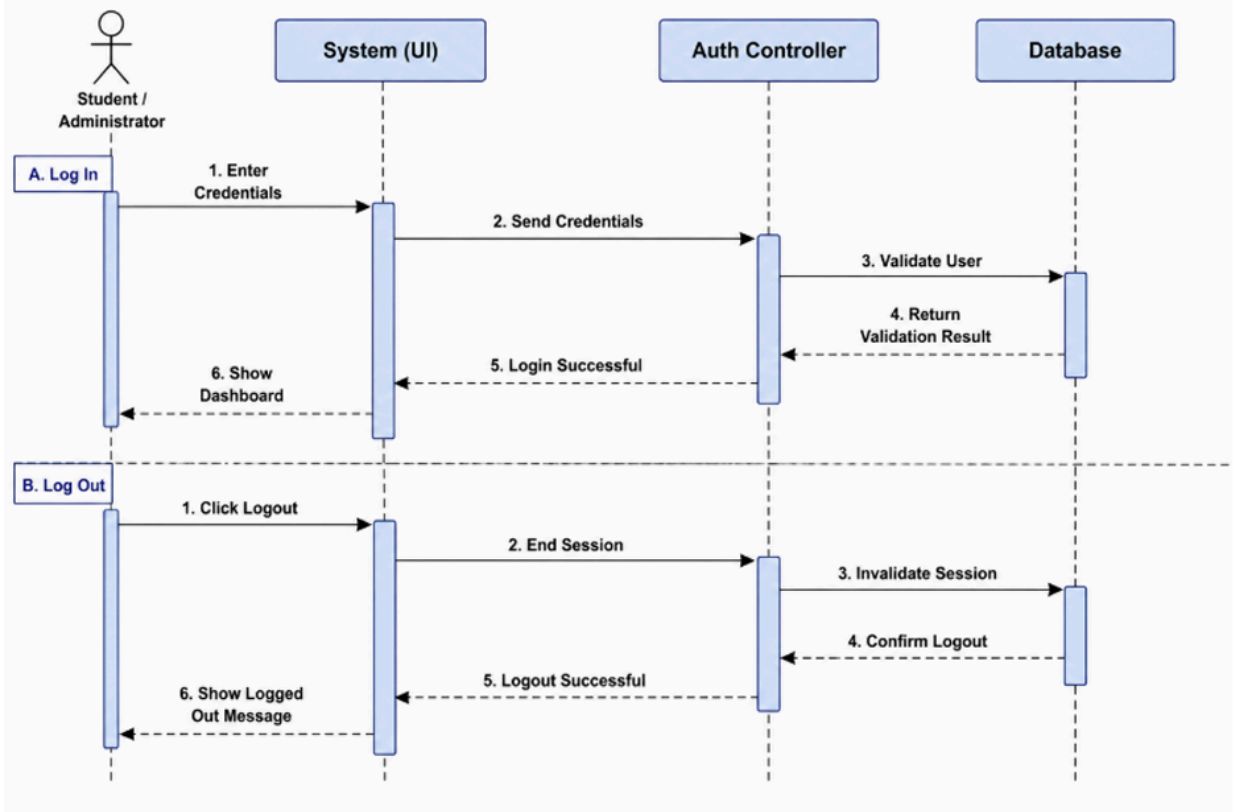


8. Sequence Diagram

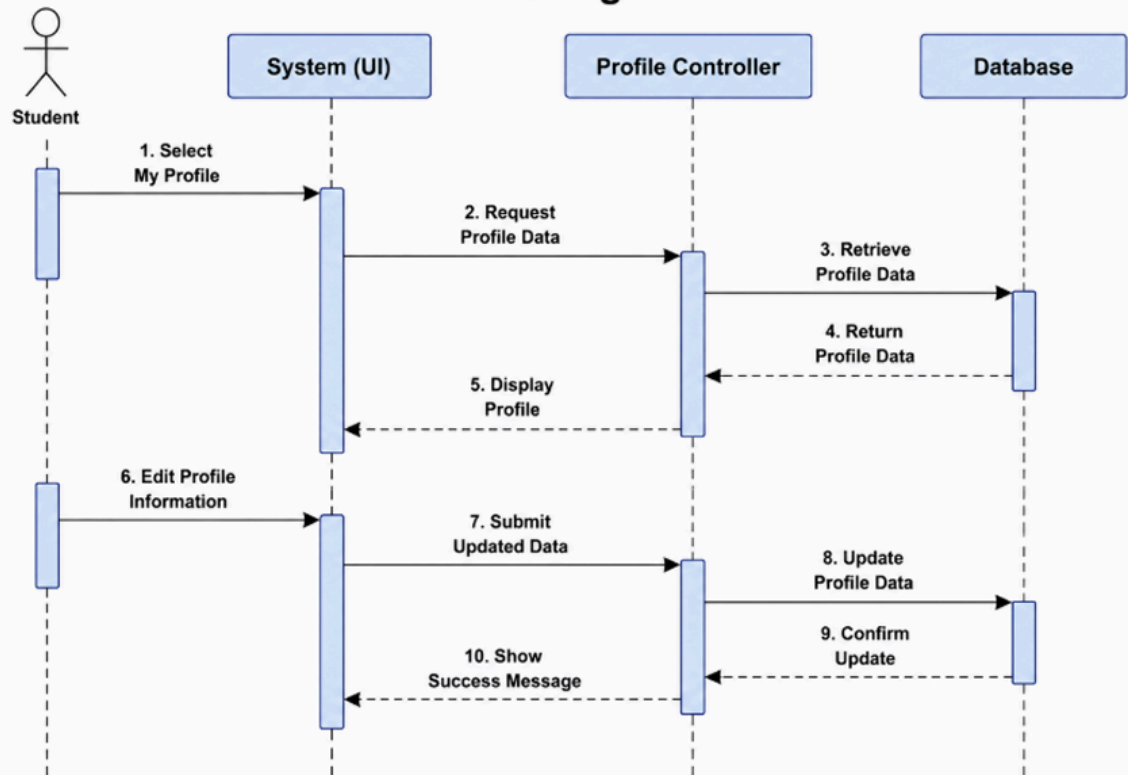
UC1: Register Account



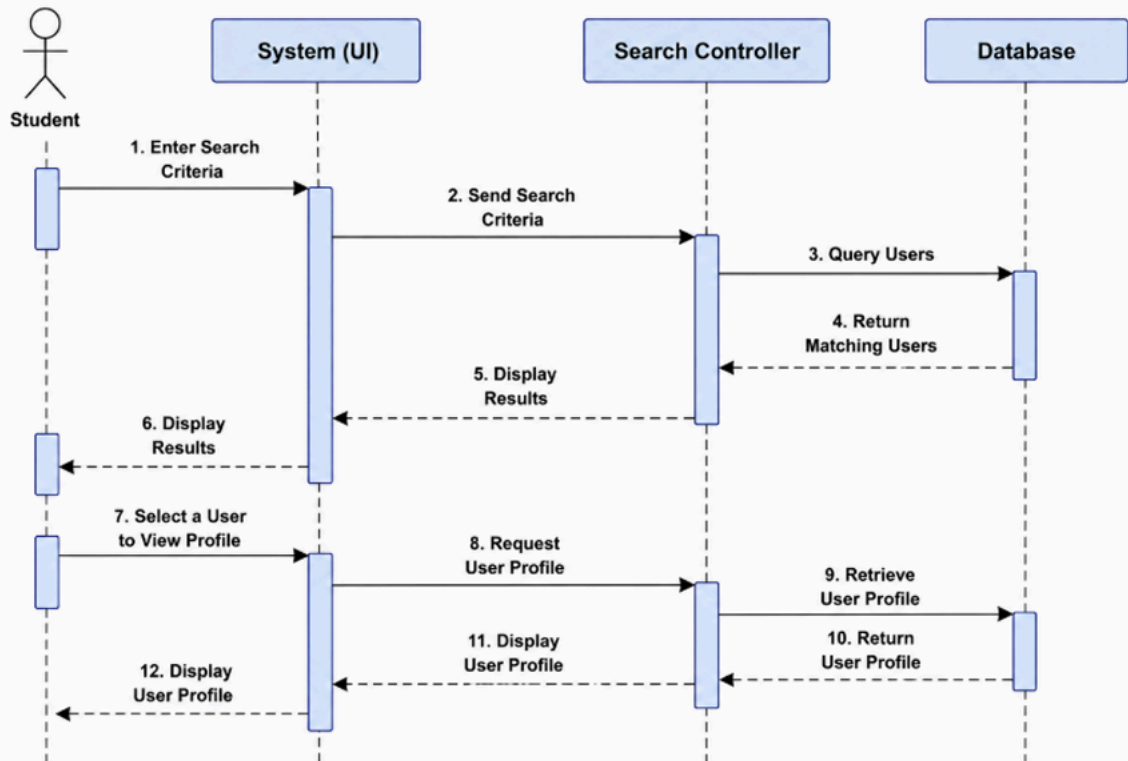
UC2: Log In / Log Out



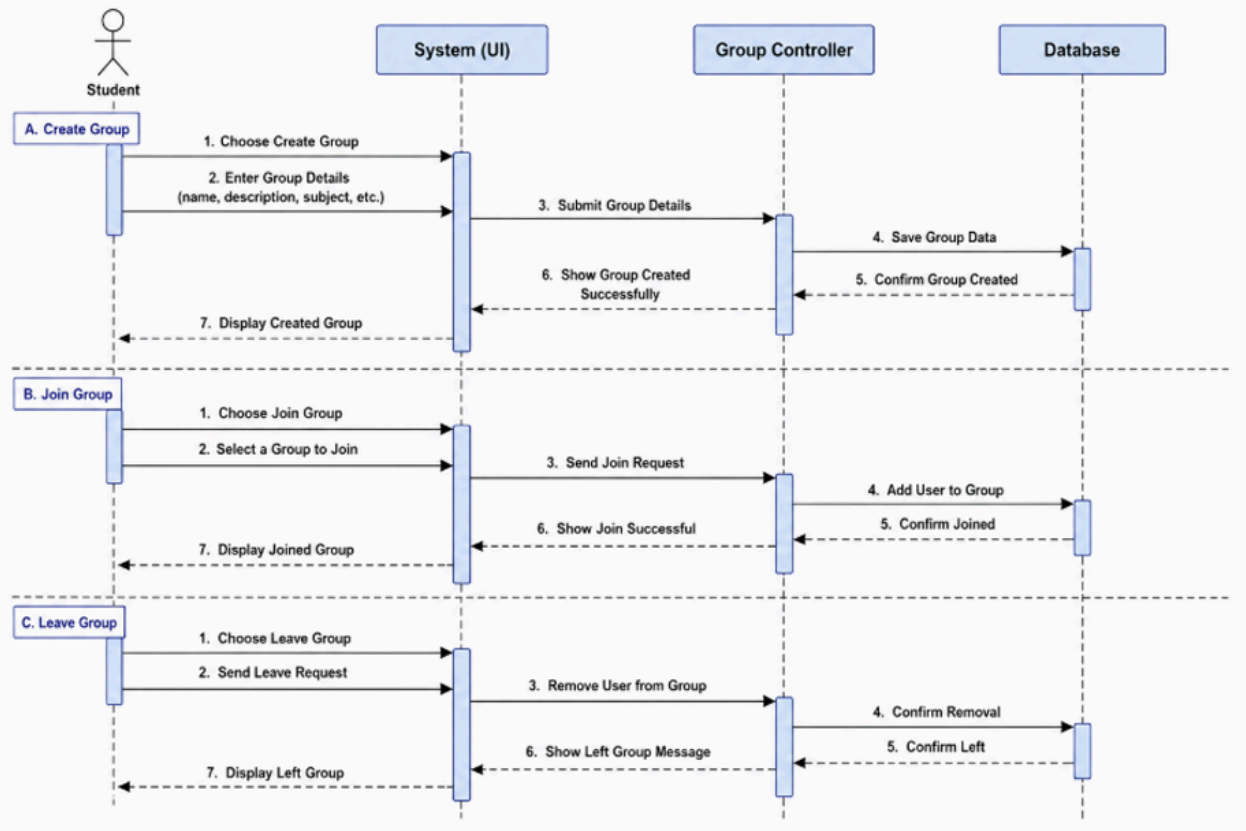
UC3: Manage Profile



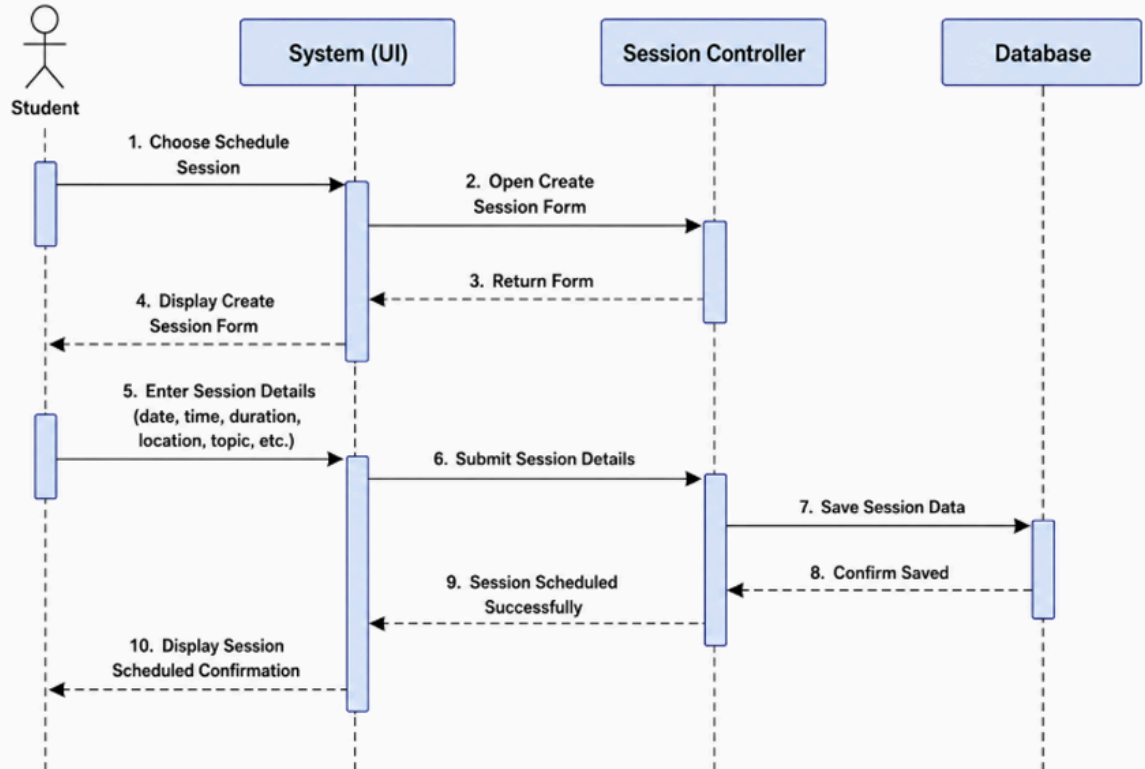
UC4: Search for Study Partners

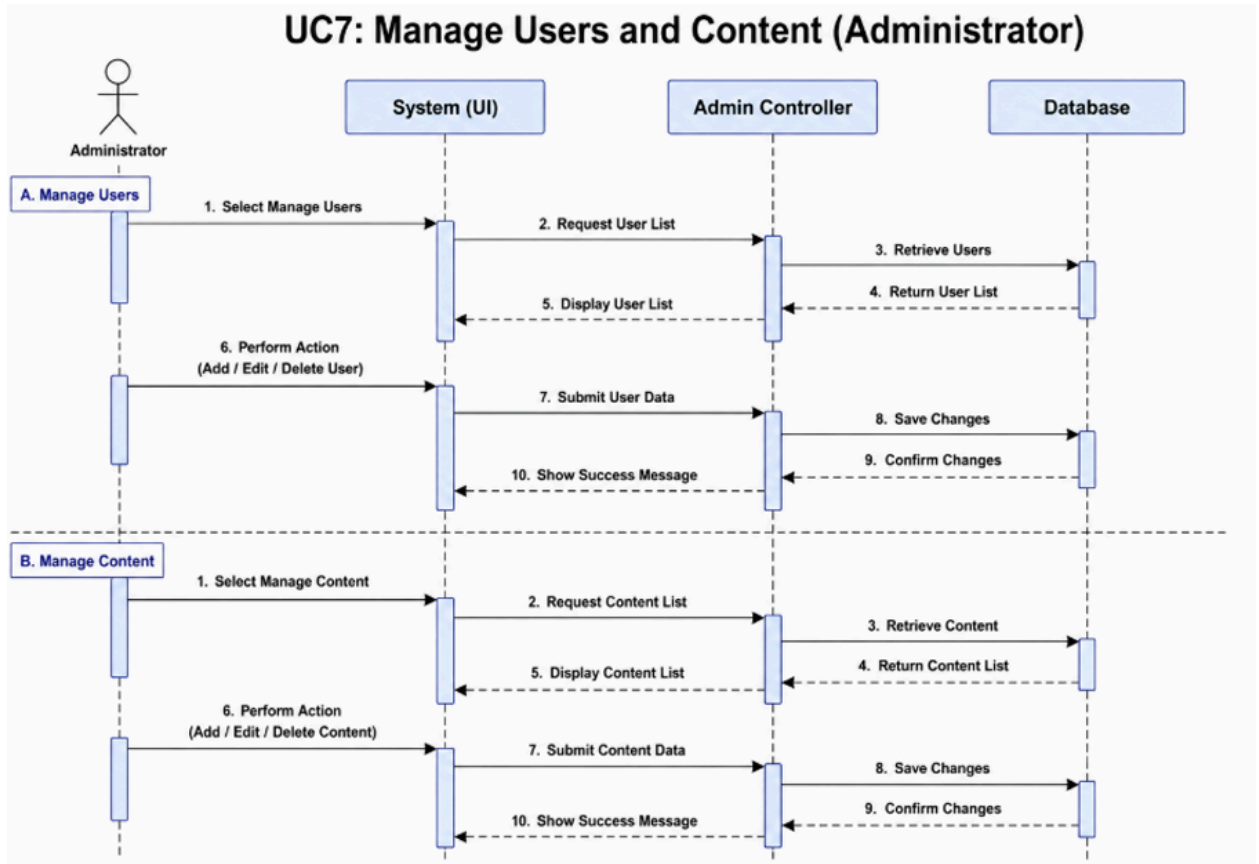


UC5: Manage Study Groups (Create / Join / Leave)

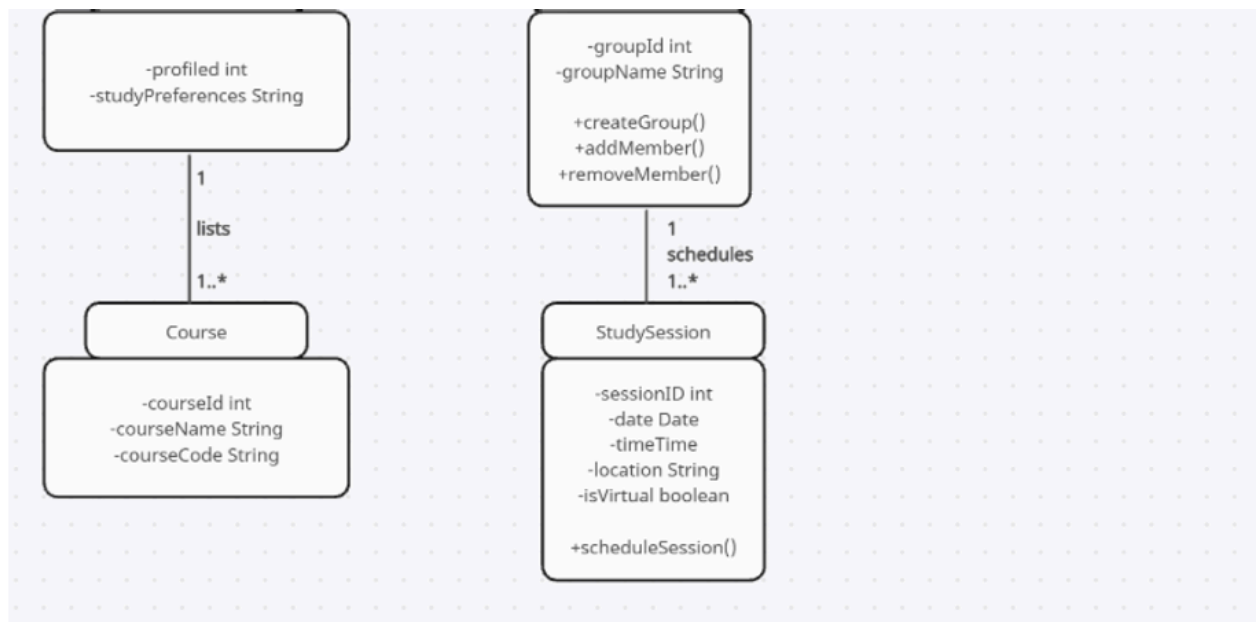
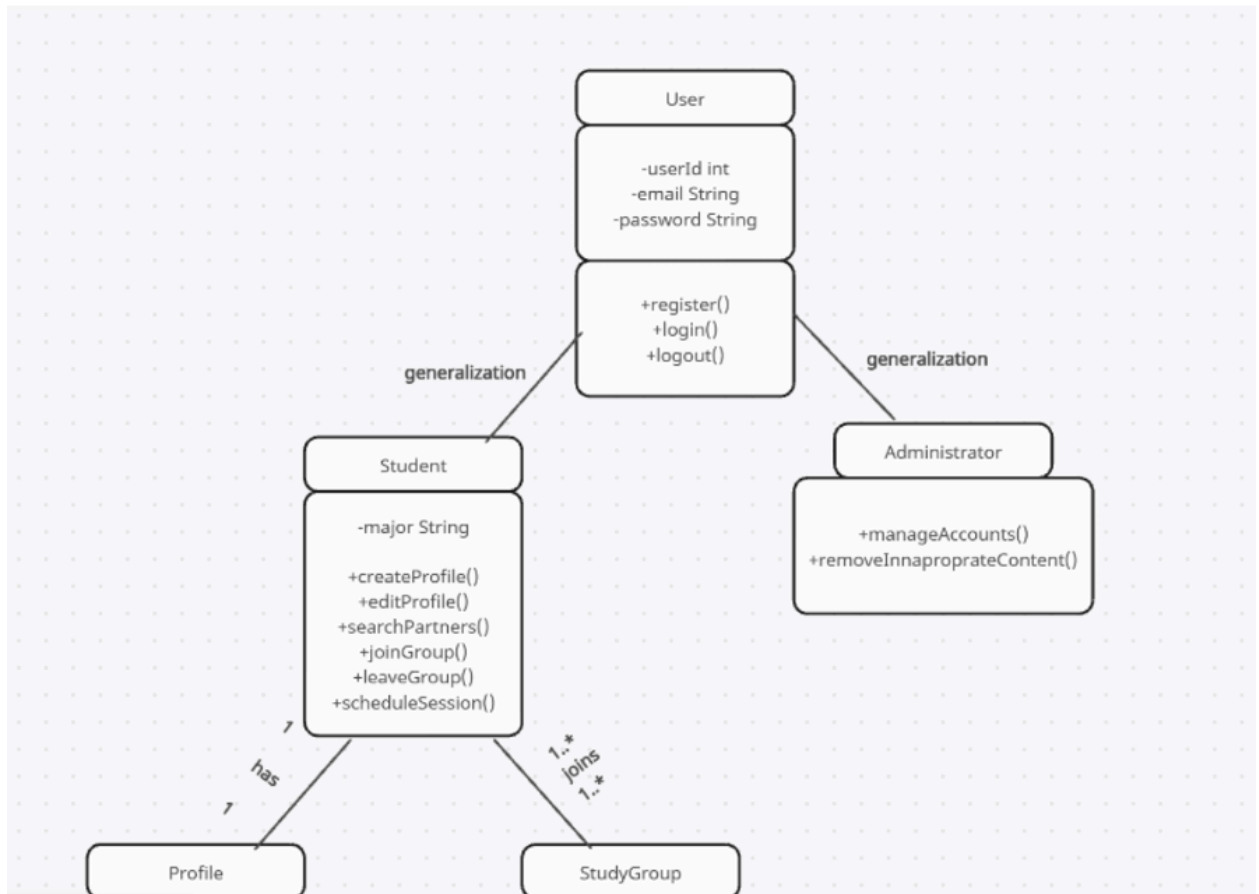


UC6: Schedule Study Sessions

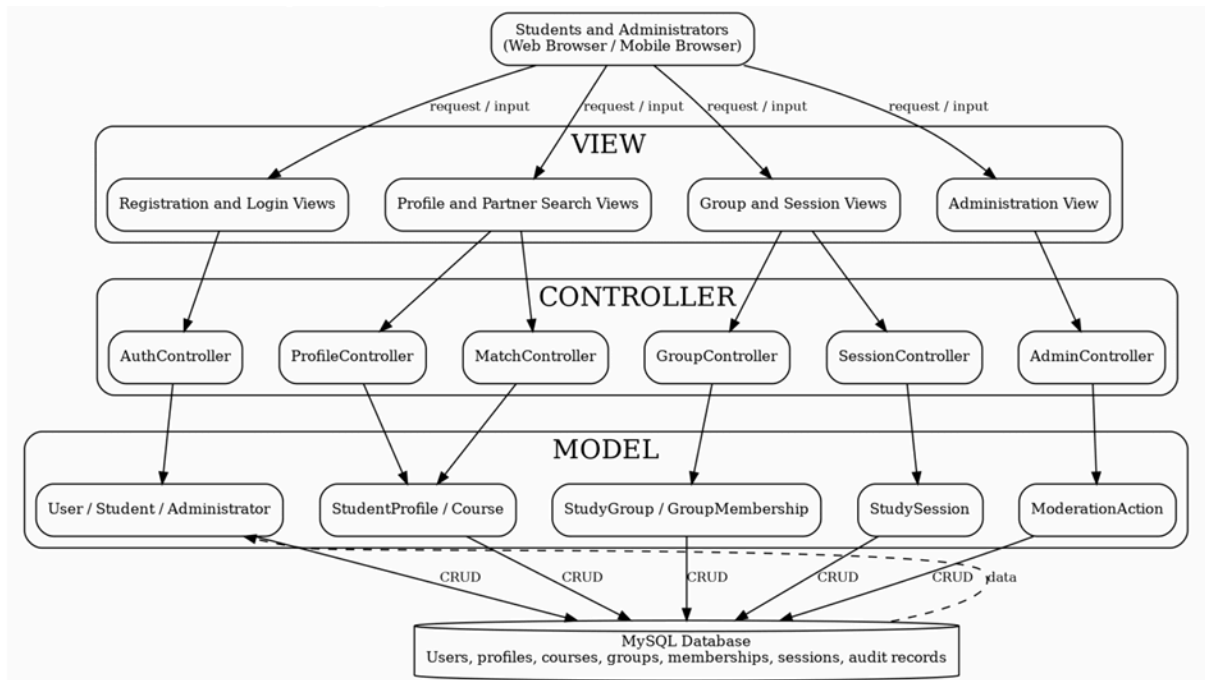




9. Class Diagram



10. Model-View-Controller Diagram



4. Project Scheduling, Cost, Effort, and Pricing Estimation

The following schedule and cost calculations are estimates based on the expected size and complexity of the Study Buddy Finder application. The estimates assume that five team members will participate throughout the project.

a. Project Scheduling

Estimated Start Date: June 14, 2026

Estimated End Date: August 13, 2026

Estimated Duration: Approximately nine weeks

The Study Buddy Finder project is expected to take approximately nine weeks to complete. This schedule provides time for requirements analysis, system design, prototype development, frontend and backend implementation, integration, testing, debugging, documentation, and preparation for the final presentation.

The project will be completed in phases so that the team can review each major component before proceeding to the next phase. This approach will also allow the team to identify and correct problems throughout development rather than waiting until the end of the project.

Weekend Schedule

Weekends will not be counted as regular project workdays. The official schedule assumes that work will be performed Monday through Friday. Team members may voluntarily work during weekends when additional time is needed to complete an assigned task or resolve a problem.

Working Hours

Each team member is expected to contribute approximately three hours per working day.

Therefore:

- Working days per week: 5 days
- Working hours per team member per day: 3 hours
- Working hours per team member per week: 15 hours
- Number of team members: 5
- Total team hours per week: 75 hours

- Estimated project duration: 9 weeks
- Total estimated project effort: 675 team-hours

The total effort is calculated as follows:

$$5 \text{ team members} \times 15 \text{ hours per week} \times 9 \text{ weeks} = 675 \text{ team-hours}$$

Proposed Project Timeline				
Project Phase	Duration	Start Date	End Date	Main Activities
Requirements Analysis	1 week	June 14, 2026	June 20, 2026	Gather requirements and define the project scope, users, features, and objectives.
System Design	1 week	June 21, 2026	June 27, 2026	Develop UML diagrams, system architecture, and database design.
UI/UX Prototype	1 week	June 28, 2026	July 4, 2026	Develop wireframes, screen designs, and navigation flows.
Backend Development	2 weeks	July 5, 2026	July 18, 2026	Implement server-side logic, database operations, user authentication, and application functions.
Frontend Development	2 weeks	July 19, 2026	August 1, 2026	Develop the user interface and connect it to the backend services.
Integration and Testing	1 week	August 2, 2026	August 8, 2026	Integrate the system components, execute tests, and correct defects.
Documentation and Final Presentation	5 days	August 9, 2026	August 13, 2026	Complete the report, user documentation, final revisions, and presentation.

b. Cost, Effort, and Pricing Estimation

i. Selected Estimation Method

The team selected the **Function Point Analysis** method to estimate the size, effort, and development cost of the Study Buddy Finder application.

Function Point Analysis estimates the size of a software system according to the functions it provides to users. Unlike an estimation based only on lines of code, Function Point Analysis may be used before the complete application has been implemented.

This method is appropriate for Study Buddy Finder because the application includes user inputs, outputs, inquiries, internal data, and interactions between different system components.

The functions evaluated include:

- Registering a user account
- Logging in and logging out
- Managing a student profile
- Searching for study partners
- Creating, joining, and leaving study groups
- Scheduling study sessions
- Managing users and content

- Retrieving and displaying student information
- Saving user, group, and session data

ii. Function Point Estimate

The following preliminary function-point estimate is based on the planned features of the system.

Function Type	Estimated Quantity	Average Weight	Estimated Contribution
External Inputs	8	4	32
External Outputs	4	5	20
External Inquiries	3	4	12
Internal Logical Files	3	10	30
External Interface Files	1	7	7
Estimated Unadjusted Function Points			101

Because Study Buddy Finder is considered a moderately complex application, an adjustment factor of approximately 1.05 is used.

$$\text{Adjusted Function Points} = 101 \times 1.05 = 106.05$$

The project is therefore estimated to contain approximately **106 function points**.

Based on the team's available schedule, the project is estimated to require approximately **675 total team-hours**, or about **6.37 hours per adjusted function point**.

675 hours ÷ 106 function points ≈ 6.37 hours per function point

iii. Estimated Development Effort

Team Member	Estimated Hours
Diego Chapa	135 hours
Raisa Reza	135 hours
Rafia Rauf	135 hours
Shobhit Kapoor	135 hours
Vladislav Kondratyev	135 hours
Total Estimated Effort	675 hours

The hours are divided equally for estimation purposes. Individual weekly workloads may vary depending on the complexity and timing of assigned responsibilities.

iv. Estimated Labor Cost

For this estimate, the team assumes an average software-development labor rate of **\$30 per hour**.

Estimated labor cost = 675 hours × \$30 per hour

Estimated labor cost = \$20,250

v. Pricing Estimate

A contingency amount of 15% will be added to the labor cost. This amount accounts for unexpected development problems, schedule delays, additional testing, design revisions, and other project risks.

Contingency = \$20,250 × 15%

Contingency = \$3,037.50

Pricing Component	Estimated Amount
Estimated Labor Cost	\$20,250.00
Contingency Allowance	\$3,037.50
Estimated Development Price	\$23,287.50

The estimated development price does not include hardware and personnel-support expenses, which are listed separately below.

c. Estimated Cost of Hardware Products

Study Buddy Finder is planned as a web-based application. Therefore, the project does not require the purchase of extensive physical equipment. Existing team computers will be used for development.

Hardware Product	Purpose	Estimated Cost
Cloud Development Server	Hosts the application and database during development and testing	\$300
Backup Storage	Stores backups of project files and system data	\$100
Networking and Testing Equipment	Supports network access and application testing	\$150
Total Estimated Hardware Cost		\$550

The hardware estimate assumes that team members already own suitable computers and have reliable internet access. The cost of personal computers is therefore not included.

d. Estimated Cost of Software Products

The team plans to use free, open-source, or educational software whenever possible.

--

Software Product	Purpose	Estimated Cost
GitHub Student Plan	Source control and team collaboration	\$0
Visual Studio Code	Software development	\$0
Java Development Kit	Application development	\$0
MySQL Community Edition	Database management	\$0
draw.io	UML and architectural diagrams	\$0
Figma Educational Plan	UI/UX prototype design	\$0
Testing Frameworks	Software and unit testing	\$0
Total Estimated Software Cost		\$0

No paid software licenses are expected to be necessary because the selected tools provide free student, community, or open-source versions.

e. Estimated Cost of Personnel

The primary personnel expense is the estimated labor cost of the five team members. Additional costs are included for training materials and post-installation support.

--

Personnel or Service	Calculation or Purpose	Estimated Cost
Five Student Developers	675 total hours × \$30 per hour	\$20,250
User Training Materials	Instructions, user guide, and training resources	\$400
Post-Installation Support	Initial maintenance and user assistance	\$850
Total Estimated Personnel Cost		\$21,500

The personnel estimate assumes that the five team members will perform project management, requirements analysis, design, implementation, testing, and documentation. No additional external employees or consultants are included.

Overall Estimated Project Cost

Cost Category	Estimated Cost
Development Labor	\$20,250.00
Hardware Products	\$550.00
Software Products	\$0.00
Training Materials	\$400.00
Post-Installation Support	\$850.00

Contingency Allowance	\$3,037.50
Overall Estimated Project Price	\$25,087.50

The overall estimated price of developing and initially supporting the Study Buddy Finder application is therefore **\$25,087.50**.

5. Test Plan

a. Unit Test Plan

Unit Under Test: MatchService.score(Candidate self, Candidate candidate)

Purpose:

This method determines whether two students should be matched based on shared courses, academic interests, study mode, and overlapping availability. It returns a compatibility score and match reasons when students share at least one course; otherwise, it returns an empty result.

Testing Objectives:

- Correctly identifies students with compatible schedules.
- Requires at least one shared course.
- Computes a valid compatibility score.
- Returns meaningful reasons explaining the match.
- Rejects incompatible candidates.

Programming Language: Java

IDE: Visual Studio Code

Testing Framework: JUnit 5

Operating System: Windows

Date Tested: 7/26/2026

b. Unit Code/Test -Unit Code:

```
public Optional<Scored> score(Candidate self, Candidate candidate) {  
    Set<String> shared = new TreeSet<>(self.courses());  
    shared.retainAll(candidate.courses());  
  
    if (shared.isEmpty()) return Optional.empty();  
  
    Set<String> commonInterests = new TreeSet<>(self.interests());  
    commonInterests.retainAll(candidate.interests());  
  
    boolean compatibleMode =  
        "FLEXIBLE".equalsIgnoreCase(self.studyMode()) ||  
        "FLEXIBLE".equalsIgnoreCase(candidate.studyMode()) ||  
        self.studyMode().equalsIgnoreCase(candidate.studyMode());  
  
    boolean overlap = hasOverlap(  
        self.availability(), candidate.availability()  
    );  
  
    int score = shared.size() * 50  
        + commonInterests.size() * 10  
        + (compatibleMode ? 10 : 0)  
        + (overlap ? 10 : 0);  
}
```

```

List<String> reasons = new ArrayList<>();
reasons.add(shared.size() + " shared course");

if (!commonInterests.isEmpty())
    reasons.add("shared academic interests");

if (compatibleMode)
    reasons.add("compatible study mode");

if (overlap)
    reasons.add("overlapping availability");

return Optional.of(new Scored(candidate, score, reasons));
}

```

JUnit Test Code:

```

@Test
void scoresStudentsWhoShareAnActiveCourseAndAvailability() {
    var ada = candidate(1L, Set.of("CS 3354"),
        Set.of("software design"), "IN_PERSON", 18, 20);

    var sam = candidate(2L, Set.of("CS 3354"),
        Set.of("software design"), "FLEXIBLE", 19, 21);

    var result = service.score(ada, sam);

    assertThat(result).isPresent();
}

```

```

    assertThat(result.orElseThrow().score()).isGreaterThanOrEqualTo(70);
    assertThat(result.orElseThrow().reasons())
        .contains("1 shared course", "overlapping availability");
}

```

@Test

```

void doesNotMatchStudentsWithoutASharedCourse() {
    var ada = candidate(1L, Set.of("CS 3354"),
        Set.of("software design"), "IN_PERSON", 18, 20);

    var lee = candidate(3L, Set.of("CS 2305"),
        Set.of("history"), "ONLINE", 18, 20);

    assertThat(service.score(ada, lee)).isEmpty();
}

```

c. Test Cases

The first test case verifies a true match. Two students, Ada and Sam, are enrolled in CS 3354 and have overlapping availability on Tuesday evening. They also have similar academic interests and compatible study preferences. The expected result is that the system returns a match with a compatibility score of at least 70 and includes reasons such as “1 shared course” and “overlapping availability.” The test passed successfully.

The second test case verifies a false match. Ada is enrolled in CS 3354, while Lee is enrolled in CS 2305. Although their availability times overlap, they do not share an active

course. The expected result is that the system does not return a match. The test passed successfully and returned an empty result.

The third test case verifies access control. An unauthenticated user attempts to access private application data. The expected result is that access is denied. The test passed successfully.

All JUnit tests completed successfully with 3 tests, 0 failures, and 0 errors.

6. Comparison

Two real-world systems with designs similar to our project are Meetup and StudyTogether connecting students from all over. Both platforms support connecting users with others who share common interests, but they differ in their target audience and matching approach. Meetup allows for the creation of groups and scheduling events of any scale. While it can be used to organize study sessions, its primary focus is on public communities and social events rather than academic collaboration. Study Buddy Finder prioritizes student to student interaction, making sure whoever you're planning to meet with will be a peer from your campus. This targeted approach increases the likelihood that matched students have common academic goals and can effectively collaborate.

StudyTogether brings learners into an online environment allowing them to teach, support, and share with one another. Although it encourages collaboration and peer support, it does not automatically match users based on academic compatibility.

StudyBuddy Finder takes your profile and matches you with people based on factors like

courses, availability, and preferred method of study. By focusing on institution-specific matching and in-person or flexible study sessions, our design emphasizes helping students build productive academic partnerships rather than simply providing a space for online interaction.

7. **Conclusion**

Study Buddy Finder was completed to help university students find compatible study partners, create study groups, schedule sessions, and communicate with approved group members. The final system includes user accounts, student profiles, course-based matching, group membership requests, study sessions, group discussions, reporting, and administrator moderation.

The main project idea remained consistent with the original plan. Some features were refined to improve privacy and security. Student email addresses and detailed availability are kept private, while group discussions and sessions are available only after a membership request is approved. Testing confirmed that compatible students can be matched and that unauthorized users cannot access private group content or administrator functions.

Future improvements may include university SSO, calendar integration, direct messaging, and permanent cloud hosting.

8. References

[1] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[2] International Function Point Users Group (IFPUG), "Function Point Counting Practices Manual," IFPUG, Westerville, OH, USA. [Online]. Available: <https://ifpug.org>

[3] GitHub, "GitHub," [Online]. Available: <https://github.com>

9. Presentation